

Part 1: Mobile Python Script to Generate Clean JSONL Logs

This script allows you or your field agents to type raw notes directly into the mobile terminal, automatically formatting and appending them to your agent_logs.jsonl file.

First, install Python on your phone's terminal app:

```
pkg install python -y
```

Next, create the Python logging script by running this exact block:

```
cat << 'EOF' > log_builder.py
import json
import datetime
import os

def add_log_line():
    log_file = os.path.expanduser("~/AgentWorkspace/agent_logs.jsonl")

    print("--- 📱 Mobile Agent Log Entry ---")
    agent_id = input("Enter Agent ID (e.g., A-102): ")
    client_name = input("Enter Client Name: ")
    status = input("Enter Status (active/pending/flagged): ")
    summary = input("Enter Note/Summary: ")

    # Structure the payload
    payload = {
        "timestamp": datetime.datetime.utcnow().isoformat() + "Z",
        "agent_id": agent_id,
        "client_name": client_name,
        "status": status,
        "interaction_summary": summary
    }

    # Ensure directory exists and append line
    os.makedirs(os.path.dirname(log_file), exist_ok=True)
    with open(log_file, "a") as f:
        f.write(json.dumps(payload) + "\n")

    print("✅ Log entry securely appended to JSONL workspace.\n")

if __name__ == "__main__":
    add_log_line()
EOF
```

```
chmod +x log_builder.py
```

To log a new note on your phone anytime, just type `python log_builder.py`.

Part 2: Injecting Custom Prompt Variables into skills.md

To make your skills.md file truly dynamic, you must declare **Prompt Variables** at the bottom of the file. This tells your AI agent how to inject real-time context (like who is using the phone, their GPS coordinates, or their current operational clearance) directly into every search it performs.

Append this configuration block directly onto the end of your existing skills.md file:

5. Dynamic Prompt Variables

Agents must look for and inject the following contextual headers into every text search execution to enforce security partitions:

```
| Variable Name | System Injection Method | Operational Rule |
| :--- | :--- | :--- |
| `{{CURRENT_AGENT}}` | System Environment Login | Restrict file access to rows
matching this Agent ID unless clearance is set to 'Admin'. |
| `{{DEVICE_LOC}}` | Mobile GPS Payload | Log latitude/longitude with every automated
"grip" execution. |
| `{{TARGET_DATE}}` | System Datetime | Auto-filter JSONL logs to only parse data lines
from the matching ISO string prefix. |
```

6. Execution Example For Agents

When executing a file search command on behalf of a mobile user, format your internal thinking block using the variables like this:

```
"User requested a file grip for Acme Corp. Current context is agent ID
{{CURRENT_AGENT}}. Compiling terminal command: grep 'Acme Corp' | grep
'{{CURRENT_AGENT}}'."
```

Next Steps for Your System

1. Run `python log_builder.py` on your phone to create your first clean data rows.
2. Run your `./sync_and_grep.sh` script to test searching through both your new logs and your updated skills.md prompt variable rules.

Would you like me to write a **master wrapper script** that bundles your sync, logging, and search operations into a simple **one-tap interactive menu** on your phone?